

Sprint 2 – Evidências de Integração com MOM

Middleware: RabbitMQ 3.13.7 | Backend: Node.js / Express
Exchange: **bahfood** | Eventos: **pedido.criado** e **pedido.status.atualizado**
Data: 22/05/2026

Sobre este documento

Este documento reúne as evidências visuais exigidas para a Sprint 2 do Projeto Integrador, comprovando o funcionamento do Middleware Orientado a Mensagens (RabbitMQ) integrado ao backend REST. As evidências estão organizadas na seguinte ordem: (1) requisições REST via Postman que disparam os eventos; (2) logs do terminal demonstrando a publicação e o consumo assíncrono das mensagens; (3) telas do RabbitMQ Management confirmando as filas, bindings e os spikes de tráfego de mensagens.

Fluxo de Eventos Demonstrado

Evento 1 – pedido.criado: Cliente faz POST em `/api/pedidos` → backend persiste no PostgreSQL e publica evento na exchange *bahfood* com routing key *pedido.criado* → PrestadorConsumer consome da *fila.prestador* e exibe os dados do novo pedido.

Evento 2 – pedido.status.atualizado: Prestador faz PATCH em `/api/pedidos/:id/status` → backend atualiza o status e publica evento com routing key *pedido.status.atualizado* → ClienteConsumer consome da *fila.cliente* e exibe a atualização para o cliente.

1. Requisições REST (Postman)

As figuras abaixo mostram as chamadas REST que disparam os eventos no MOM. Cada requisição bem-sucedida aciona o produtor, que publica a mensagem na exchange do RabbitMQ.

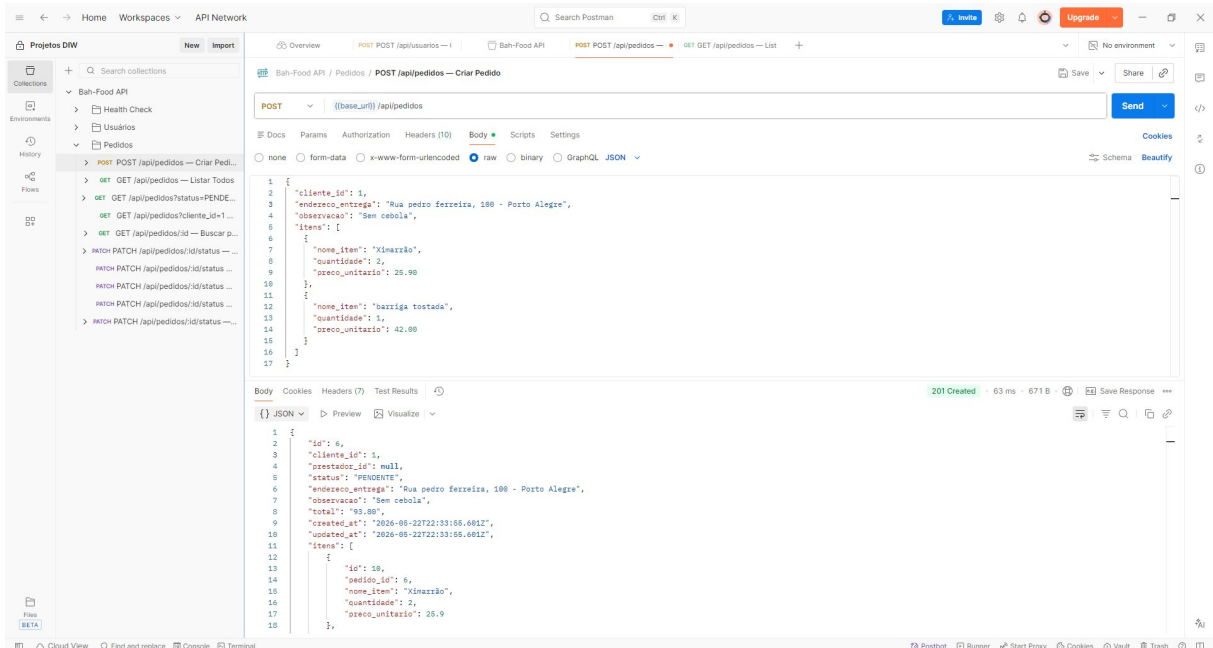


Figura 1 – Postman: POST /api/pedidos — Criação de Pedido (201 Created)

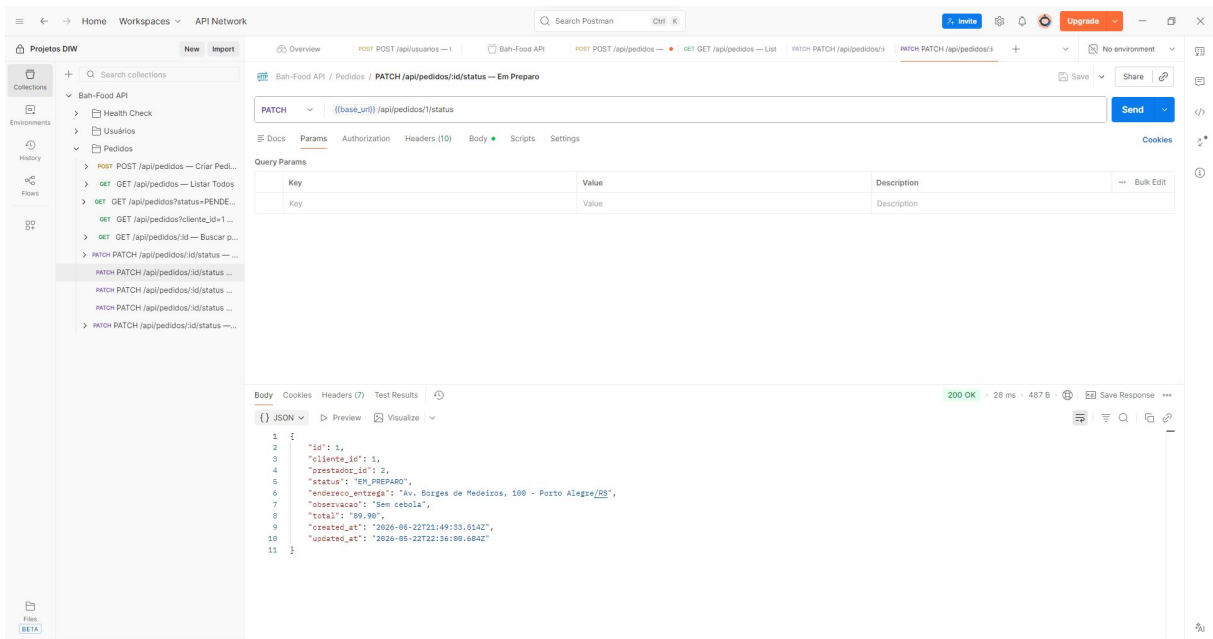


Figura 2 – Postman: PATCH /api/pedidos/id/status — Atualização de Status (200 OK)

2. Logs do Terminal – Publicação e Consumo Assíncrono

Os logs abaixo comprovam a assincronicidade real do sistema: o Producer publica o evento e, sem qualquer chamada REST direta, o Consumer correspondente processa a mensagem da fila.

```
PS C:\Users\Arthur\Faculdade\Trabalho Bah food\Trabalho-Bah-food\src> npm run dev
◊ injected env (7) from .env // tip: override existing { override: true }
◊ injected env (8) from .env // tip: custom filepath { path: '/custom/path/.env' }
[DB] Conectado ao PostgreSQL
[RabbitMQ] Conectado ao broker
[PrestadorConsumer] Aguardando eventos em "fila.prestador" (routing key: pedido.criado)
[ClienteConsumer] Aguardando eventos em "fila.cliente" (routing key: pedido.status.atualizado)
[HTTP] Servidor rodando em http://localhost:3000
[Producer] Evento publicado → [pedido.criado] {
  evento: 'pedido.criado',
  timestamp: '2026-05-22T19:31:25.962Z',
  pedido_id: 6,
  cliente_id: 1,
  endereco: 'Rua pedro ferreira, 100 - Porto Alegre',
  total: '93.80',
  itens: [
    {
      id: 10,
      pedido_id: 6,
      nome_item: 'Ximarrão',
      quantidade: 2,
      preco_unitario: 25.9
    },
    {
      id: 11,
      pedido_id: 6,
      nome_item: 'barriga tostada',
      quantidade: 1,
      preco_unitario: 42
    }
  ]
}

[PRESTADOR] Novo pedido recebido!
ID do Pedido : 6
Cliente ID : 1
Endereço : Rua pedro ferreira, 100 - Porto Alegre
Total : R$ 93.80
Itens : [{"id":10,"pedido_id":6,"nome_item":"Ximarrão","quantidade":2,"preco_unitario":25.9},{"id":11,"pedido_id":6,"nome_item":"barriga tostada","quantidade":1,"preco_unitario":42}]
Recebido em : 2026-05-22T19:31:25.962Z
```

Figura 3 – Terminal: Evento pedido.criado publicado e consumido pelo [PRESTADOR]

```
File Edit Selection View Go Run Terminal Help
Trabalho-Bah-food
.env
1 DB_HOST=localhost
2 DB_PORT=5433
3 DB_NAME=bahfood
4 DB_USER=postgres
5 DB_PASSWORD=postgres
6 PORT=3000
7 RABBITMQ_URL=amqp://guest:guest@localhost:5673
8

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\Arthur\Faculdade\Trabalho Bah food\Trabalho-Bah-food\src> npm run dev

id: 10,
pedido_id: 6,
nome_item: 'Ximarrão',
quantidade: 2,
preco_unitario: 25.9
},
{
id: 11,
pedido_id: 6,
nome_item: 'barriga tostada',
quantidade: 1,
preco_unitario: 42
}
}

[PRESTADOR] Novo pedido recebido!
ID do Pedido : 6
Cliente ID : 1
Endereço : Rua pedro ferreira, 100 - Porto Alegre
Total : R$ 93.80
Itens : [{"id":10,"pedido_id":6,"nome_item":"Ximarrão","quantidade":2,"preco_unitario":25.9},{"id":11,"pedido_id":6,"nome_item":"barriga tostada","quantidade":1,"preco_unitario":42}]
Recebido em : 2026-05-22T19:31:25.962Z
[Producer] Evento publicado → [pedido.status.atualizado] {
  evento: 'pedido.status.atualizado',
  timestamp: '2026-05-22T19:33:32.362Z',
  pedido_id: 1,
  cliente_id: 1,
  prestador_id: 2,
  status_novo: 'EM_PREPARO'
}

[CLIENTE] Status do pedido atualizado!
ID do Pedido : 1
Novo Status : EM_PREPARO
Prestador ID : 2
Atualizado em : 2026-05-22T19:33:32.362Z
```

Figura 4 – Terminal: Evento pedido.status.atualizado publicado e consumido pelo [CLIENTE]

3. RabbitMQ Management – Filas e Tráfego de Mensagens

As telas do RabbitMQ Management Console confirmam: filas duráveis criadas, consumidores ativos (status *up*), bindings corretos para cada routing key e os spikes no gráfico de Message Rates comprovando o tráfego real de mensagens.

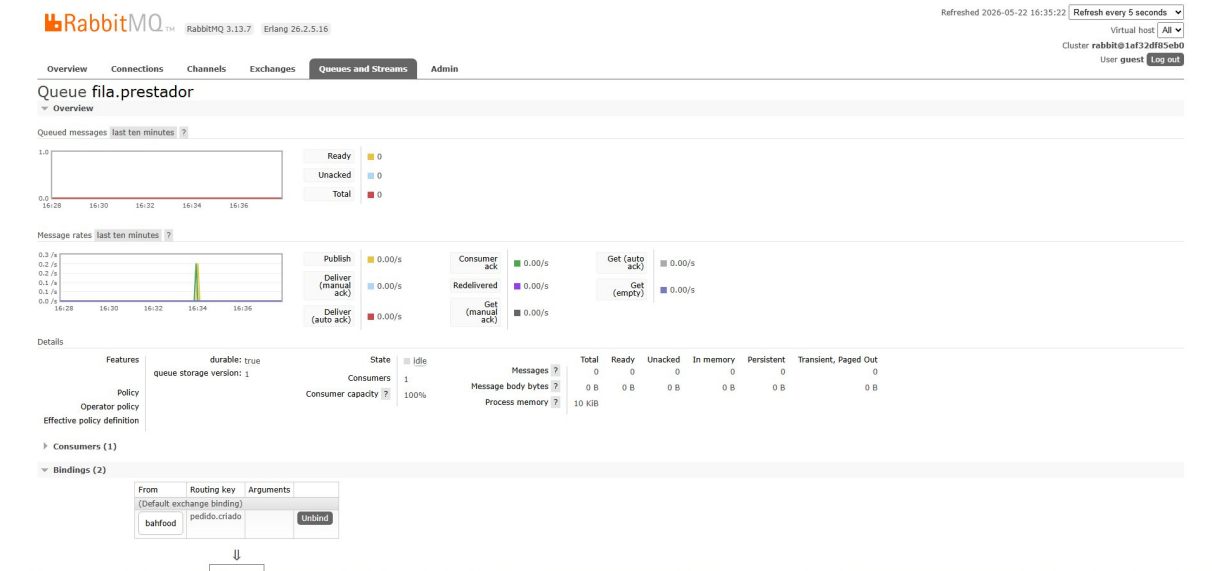


Figura 5 – RabbitMQ Management: fila.prestador com spike de mensagens e consumidor ativo

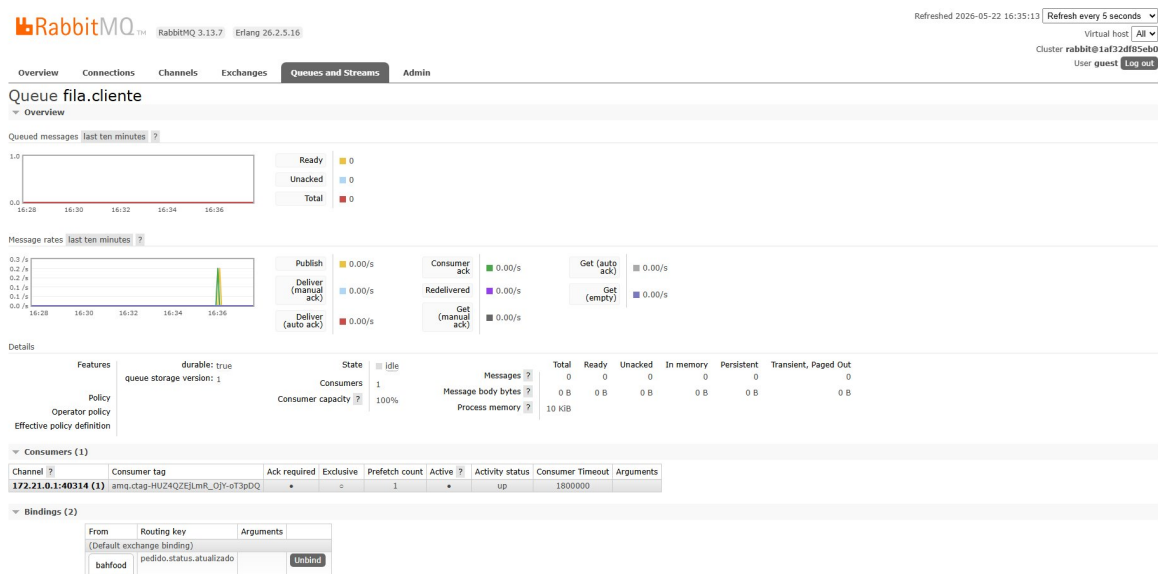


Figura 6 – RabbitMQ Management: fila.cliente com spike de mensagens e consumidor ativo